

Member 2 — UI/UX Code-Then-Commit Workflow

Owner: [Real name / Reg. No.] · **Branch:** `feature/ui-refresh`

Main work: Jinja/HTML templates and CSS.

This is an implementation guide for Member 2 to follow on the team's real repository. The app already contains templates and a stylesheet, so the snippets below are incremental improvements to those existing files—not a claim that the whole interface was built from zero. Compare with the current branch before editing. Keep only changes that run correctly in this project.

Sports-interface and event-background scope

For the sports-first redesign, Member 2 owns the visible matchday hero, scoreboard card, event-category accents, dark stadium palette and admin-facing selector styling. Coordinate with Member 1: Member 1 owns the stored `background_theme`, Auto-to-sport resolution, next-event data and create/edit persistence; Member 2 owns how that data is shown in templates and styled in CSS. The homepage hero should use the earliest upcoming event's effective theme, and the admin event form should make Auto vs preset override clear. Do not add image uploads or change database/routes as a UI-only task.

Relevant display files are `templates/index.html`, `templates/admin/event_form.html`, `templates/admin/dashboard.html`, `templates/_event_card.html`, `templates/event_detail.html`, and `static/css/style.css`.

Exact files this plan edits

```
templates/base.html
static/css/style.css
templates/index.html
templates/events.html
templates/_event_card.html
templates/event_detail.html
templates/registration_form.html
templates/registration_success.html
templates/admin/login.html
templates/admin/dashboard.html
templates/admin/event_form.html
templates/admin/registrations.html
templates/errors/400.html
templates/errors/403.html
templates/errors/404.html
templates/errors/500.html
```

`static/js/main.js` is not part of these chunks. Do not change Flask routes, models, form fields or endpoints. Keep Jinja conditions/loops, form POST methods, and all CSRF fields.

Start once — pull, then create the branch

Use the real repository URL supplied by the team. Clone only if the project is not already on the computer.

```
git clone https://github.com/<OWNER>/<REPOSITORY>.git
cd <REPOSITORY>
git switch main
git pull --ff-only origin main
git switch -c feature/ui-refresh
```

If the folder is already cloned, start inside it, run `git status`, switch to `main`, pull, and then create the branch. Create this branch once; all six chunks below stay on `feature/ui-refresh`.

At the beginning, run the app and inspect the existing home page, event list/detail, registration form and admin screens. Record genuine baseline screenshots/notes if useful. Do not make a commit just for

opening the app or for creating a time marker.

How to use each code chunk

For each chunk: edit only the files listed; run the check stated; inspect the diff; commit that completed code slice; push the same branch. The first push uses `-u`; later pushes do not. The snippets show the intended change—preserve surrounding project code and do not replace whole files with these fragments.

Chunk 1 — Show the active navigation page

Files: `templates/base.html`, `static/css/style.css`

Purpose: make it clear which main area the visitor is viewing, including when they are on an event detail or registration page.

Code change A — update the two public navigation links in `templates/base.html`

Replace the current Home and Events anchors with:

```
<a href="{{ url_for('main.home') }}"
    {% if request.endpoint == 'main.home' %}aria-current="page"{% endif %}>Home</a>
<a href="{{ url_for('main.events_list') }}"
    {% if request.endpoint in ['main.events_list', 'events.detail', 'events.register'] %}aria-current="page"{% endif %}>Events</a>
```

Code change B — update the admin dashboard link in the same file

Replace the existing Admin dashboard anchor with:

```
<a href="{{ url_for('admin.dashboard') }}"
    {% if request.endpoint and request.endpoint.startswith('admin.') %}aria-current="page"{% endif %}>Admin
dashboard</a>
```

Code change C — add the active state near the navigation rules in

`static/css/style.css`

```
.site-nav > a[aria-current="page"]:not(.nav-cta) {
  color: var(--forest);
  border-bottom-color: var(--clay);
}
```

Check: open Home, Events, one event detail page and a registration page; the correct nav item is highlighted. Check the signed-out and admin/demo navigation too.

Commit after this code is in and checked:

```
git add templates/base.html static/css/style.css
git diff --cached
git commit -m "ui: highlight the current navigation page"
git push -u origin feature/ui-refresh
```

Chunk 2 — Improve event-list landmarks and keyboard feedback

Files: `templates/index.html`, `templates/events.html`, `templates/_event_card.html`, `static/css/style.css`

Purpose: make the public event pages easier to navigate with assistive technology and keyboard focus.

Code change A — name the home page upcoming-events section in

`templates/index.html`

Change the opening section and its heading to:

```
<section class="container schedule-section" id="upcoming" aria-labelledby="upcoming-title">
  <div class="section-heading schedule-heading">
    <div>
      <p class="eyebrow eyebrow-dark"><span class="eyebrow-rule"></span> On the calendar</p>
      <h2 id="upcoming-title">Upcoming events</h2>
    </div>
  </div>
</section>
```

Keep the rest of the section and the Jinja event loop unchanged.

Code change B — name the events-page heading in `templates/events.html`

```
<section class="container page-heading" aria-labelledby="events-title">
  <p class="eyebrow eyebrow-dark"><span class="eyebrow-rule"></span> The campus calendar</p>
  <h1 id="events-title">All sports events</h1>
</section>
```

Code change C — make the repeated event action link specific in `_event_card.html`

Replace its current `View event` link with:

```
<a class="event-row-link"
  href="{{ url_for('events.detail', slug=event.slug) }}"
  aria-label="View event: {{ event.name }}">
  View event <span aria-hidden="true">↗</span>
</a>
```

Code change D — show which event row contains keyboard focus in `static/css/style.css`

Add beside the existing `.event-row:hover` rule:

```
.event-row:focus-within {
  background: var(--surface);
  box-shadow: inset 3px 0 var(--clay);
}
```

Check: use Tab/Shift+Tab to move through event links; verify the row focus cue follows the focused link. Check the home page's empty state and events list with real seed data.

Commit after this code is in and checked:

```
git add templates/index.html templates/events.html templates/_event_card.html static/css/style.css
git diff --cached
git commit -m "ui: improve event list landmarks and keyboard feedback"
git push origin feature/ui-refresh
```

Chunk 3 — Use semantic dates and highlight the registration choice

Files: `templates/event_detail.html`, `static/css/style.css`

Purpose: expose event dates as machine-readable dates and give keyboard users feedback while choosing participant or volunteer registration.

Code change A — wrap the event date in a `<time>` element

In the Event date fact row in `event_detail.html`, replace the current `<strong>` content with:

```
<strong>
  <time datetime="{{ event.event_date.isoformat() }}">
    {{ event.event_date.strftime('%A, %d %B %Y') }}
  </time>
</strong>
```

Code change B — wrap the registration deadline in a `<time>` element

In the Registration deadline fact row, use:

```
<strong>
  <time datetime="{{ event.registration_deadline.isoformat() }}">
    {{ event.registration_deadline.strftime('%d %B %Y') }}
  </time>
</strong>
```

Code change C — add a focus-within treatment in `static/css/style.css`

Add after the existing `.registration-option` rule:

```
.registration-option:focus-within {
  background: var(--forest-wash);
}
```

Do not change the existing Jinja checks for a closed event or full participant/volunteer capacity.

Check: view an open event and an event where registration is closed/full. Confirm the exact same Register/closed/full actions still appear and go to the existing endpoints. Tab onto each registration button/link and confirm its row highlight.

Commit after this code is in and checked:

```
git add templates/event_detail.html static/css/style.css
git diff --cached
git commit -m "ui: clarify event dates and registration choices"
git push origin feature/ui-refresh
```

Chunk 4 — Link form errors to the fields that need attention

Files: `templates/registration_form.html`, `templates/registration_success.html`, `static/css/style.css`

Purpose: make invalid submissions easier to correct and identify the success page for screen readers.

Code change A — replace the generic error message in `registration_form.html`

Replace the current `{% if form.errors %}` summary with:

```
{% if form.errors %}
<div class="form-error-summary" role="alert">
  <p>Please check these fields:</p>
  <ul>
    {% for field in form %}
      {% for error in field.errors %}
        <li><a href="#{{ field.id }}">{{ field.label.text }}: {{ error }}</a></li>
      {% endfor %}
    {% endfor %}
  </ul>
</div>
{% endif %}
```

This uses the field IDs WTForms already renders. Keep `form.hidden_tag()`, all existing field names, error spans, and the participant/volunteer conditional.

Code change B — label the registration success section

In `registration_success.html`, change the opening section to:

```
<section class="container success-wrap" aria-labelledby="registration-success-title">
```

Give the participant success `<h1>` this ID:

```
<h1 id="registration-success-title">Registration successful!</h1>
```

Give the volunteer success `<h1>` the same ID (only one of these two headings is rendered at a time):

```
<h1 id="registration-success-title">Volunteer registration successful!</h1>
```

Code change C — style the linked error list in `static/css/style.css`

Add near the existing `.form-error-summary` rules:

```
.form-error-summary p {
  margin: 0 0 5px;
  color: inherit;
  font-size: inherit;
}
.form-error-summary ul {
  margin: 0;
  padding-left: 20px;
}
.form-error-summary a {
  color: inherit;
  text-decoration: underline;
  text-underline-offset: 2px;
}
```

Check: submit a blank participant form and a blank volunteer form. Each error link should move focus/navigation to its matching input; existing valid submissions should still display the correct registration type and registration ID.

Commit after this code is in and checked:

```
git add templates/registration_form.html templates/registration_success.html static/css/style.css
git diff --cached
git commit -m "ui: link registration errors to their fields"
git push origin feature/ui-refresh
```

Chunk 5 — Label admin sections/tables and improve small-screen actions

Files: `templates/admin/login.html`, `templates/admin/dashboard.html`, `templates/admin/event_form.html`, `templates/admin/registrations.html`, `static/css/style.css`

Purpose: provide useful section names for assistive technology, identify each table, and make review actions easier to tap on a phone.

Code change A — connect headings to their admin sections

In `admin/login.html`, add `aria-labelledby` to the section and an ID to its existing heading:

```
<section class="container admin-auth-page" aria-labelledby="admin-login-title">
```

```
<h1 id="admin-login-title">Admin sign in</h1>
```

In `admin/dashboard.html`, add `aria-labelledby="admin-dashboard-title"` to the outer `.admin-page` section and `id="admin-dashboard-title"` to its existing Event management `<h1>`.

In `admin/event_form.html`, add `aria-labelledby="event-form-title"` to the `.admin-form-card` div and `id="event-form-title"` to its existing `<h1>{{ heading }}</h1>`.

In `admin/registrations.html`, add `aria-labelledby="registration-review-title"` to the outer `.admin-page` section and `id="registration-review-title"` to the existing event-name `<h1>`.

Code change B — add captions to the admin tables

Immediately after the opening event table tag in `admin/dashboard.html`, add:

```
<caption class="sr-only">Events, registration totals and admin actions</caption>
```

Immediately after the opening Participants table tag in `admin/registrations.html`, add:

```
<caption class="sr-only">Participant registrations for {{ event.name }}</caption>
```

Immediately after the opening Volunteers table tag, add:

```
<caption class="sr-only">Volunteer registrations for {{ event.name }}</caption>
```

Code change C — make contact links wrap and decision buttons easier to tap

Add these rules inside the existing `@media (max-width: 700px)` block near the bottom of `static/css/style.css`:

```
.registrations-table td[data-label="Contact"] a {
  overflow-wrap: anywhere;
}
.decision-button {
  min-height: 36px;
}
```

Do not change registration data, approval/rejection URLs, POST methods or CSRF inputs.

Check: inspect admin dashboard, login, create/edit event, participant registrations and volunteer registrations. At mobile width, long email addresses must not overflow and decision buttons must remain usable. Confirm each table has a meaningful caption in the accessibility tree.

Commit after this code is in and checked:

```
git add templates/admin/login.html templates/admin/dashboard.html templates/admin/event_form.html templates/
admin/registrations.html static/css/style.css
git diff --cached
git commit -m "ui: improve accessible admin tables and mobile actions"
git push origin feature/ui-refresh
```

Chunk 6 — Label error pages and honor reduced-motion settings

Files: `templates/errors/400.html`, `templates/errors/403.html`, `templates/errors/404.html`, `templates/errors/500.html`, `static/css/style.css`

Purpose: make error-page headings explicit landmarks and respect the user's motion preference.

Code change A — label each error section

In each of the four error templates, add `aria-labelledby="error-title"` to the opening `.error-page` section. Add `id="error-title"` to that page's existing `<h1>`. Example for `404.html`:

```
<section class="container error-page" aria-labelledby="error-title">
  <span class="error-code">404</span>
  <h1 id="error-title">We can't find that page</h1>
```

Use the existing heading text for 400, 403 and 500; do not change their return-home links.

Code change B — respect reduced-motion preference in `static/css/style.css`

Add at the end of the stylesheet:

```
@media (prefers-reduced-motion: reduce) {
  html {
    scroll-behavior: auto;
  }

  *,
  *::before,
  *::after {
    scroll-behavior: auto !important;
    transition-duration: 0.01ms !important;
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
  }
}
```

Check: visit an actual 404 page and any available error pages. Turn on the operating system/browser reduced-motion setting and confirm smooth scrolling/transitions are reduced. Recheck public and admin pages at ~375 px, ~768 px and ~1280 px.

Commit after this code is in and checked:

```
git add templates/errors/400.html templates/errors/403.html templates/errors/404.html templates/errors/500.
html static/css/style.css
git diff --cached
git commit -m "ally: label error pages and respect reduced motion"
git push origin feature/ui-refresh
```

Between chunks — keep the same branch

After the first `git push -u`, each later completed chunk uses `git push origin feature/ui-refresh`. If team changes land on `main` before the next chunk, first commit/stash any current work, then:

```
git fetch origin
git merge origin/main
```

Resolve and test any conflicts before continuing. Do not make a new branch for each chunk. If code was not changed, do not make an empty commit. Use actual dates, hashes, tests and screenshots only; never backdate commits or report unrun checks.

Finish — review, merge, pull

After Chunk 6, run the complete UI flows with Member 3, inspect the final diff and push the branch. Open a pull request from `feature/ui-refresh` to `main`, list files/features and checks actually completed, and ask a teammate to review. Address actual review comments in a new commit and push again. Merge on GitHub only after approval. Then update the local main branch:

```
git switch main
git pull --ff-only origin main
```

Capture genuine screenshots from the running app for presentation evidence. The code samples are a plan for Member 2 to implement; they are not evidence that those changes or commits have already happened.